

Lecture 29: Diffusion III

Sampling, and why the noise comes back

Some panels are stills from Welch Labs / 3Blue1Brown (2025).

We can train. Can we generate?

L28 ended with a trained network $\varepsilon_{\theta}(\mathbf{x}_t, t)$ that answers one question: *given this noisy image and this noise level, which noise was added?*

Generation should now be mechanical. Start from $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, ask the network what the noise is, subtract some of it, repeat.

It is almost that. But the actual sampler in the DDPM paper does something that looks like a typo the first time you read it, and today we find out why it is not.

Outline

The sampler, and a question

What the model actually learned

Why the noise comes back

Deterministic sampling

The sampler, and a question

Read the algorithm carefully. Something in it should bother you.

Algorithm 1 Training

```
1: repeat  
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$   
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$   
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
5:   Take gradient descent step on  
        $\nabla_{\theta} \left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$   
6: until converged
```

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
2: for  $t = T, \dots, 1$  do  
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$   
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$   
5: end for  
6: return  $\mathbf{x}_0$ 
```

The DDPM sampler

The two algorithms from Ho et al. (2020), side by side on the previous slide.

Algorithm 1 is training - the loop we wrote in L28.

Algorithm 2 is sampling, and it is the one to read carefully.

Reading the sampling algorithm, one step:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$$

The bracket is exactly μ_{θ} from L28 - the model's best guess at where the previous image was.

Then there is $+ \sigma_t \mathbf{z}$, with $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ drawn fresh, and $\sigma_t^2 = \beta_t$ as fixed in L28. **The sampler adds new random noise at every step.**

Hold that thought

Predict first. Our network was trained for one job: remove noise. The sampler then puts noise *back*, a thousand times over.

What happens if we simply delete that term?

Most people's first guess is "nothing much, maybe slightly less variety." The truth is more interesting, and it tells us what the model actually learned.

We will answer this in Section 3. First we need a picture of what ε_θ has learned - and for that we shrink images down to two pixels.

By hand: one step of the sampler

Take the constant $\beta = 0.02$ from L27, so $\alpha_t = 0.98$ and $\bar{\alpha}_t = 0.98^t$. We are at $t = 100$. One pixel of $\mathbf{x}_{100}$ reads 1.300, and the network predicts $\varepsilon_\theta = 1.150$ for it.

$$\begin{array}{ll} \bar{\alpha}_{100} = 0.98^{100} & = 0.13262 \\ \sqrt{1 - \bar{\alpha}_{100}} & = 0.93133 \\ \beta_t / \sqrt{1 - \bar{\alpha}_t} & = 0.02 / 0.93133 = 0.02147 \\ \mathbf{x}_t - 0.02147 \times 1.150 & = 1.27530 \\ \div \sqrt{0.98} & = 1.28825 \quad (\text{model's move}) \\ \sigma_t = \sqrt{\beta_t} & = 0.14142 \\ \text{draw } \mathbf{z} = -0.30, \text{ add } \sigma_t \mathbf{z} & = -0.04243 \\ \hline \mathbf{x}_{99} & = \mathbf{1.246} \end{array}$$

Look at the two moves.

The model shifted the pixel by $1.288 - 1.300 = -\mathbf{0.012}$.

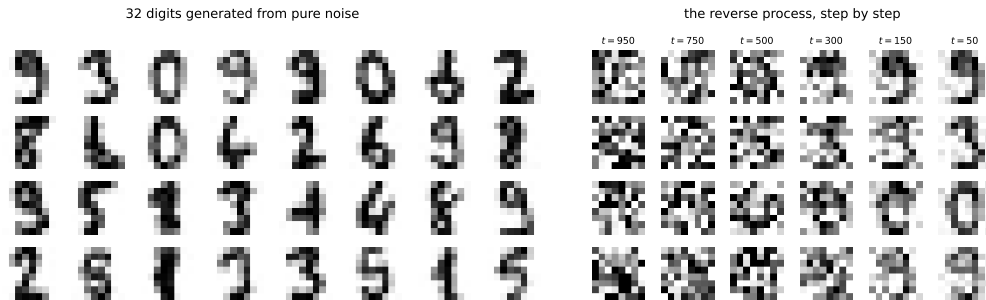
The noise shifted it by $-\mathbf{0.042}$ - about **3.5 times further**.

At this noise level the random term is not a small correction on top of the model's answer. It *dominates* the step.

Which makes the question on the last slide sharper, not softer: why would you let noise do most of the moving in a process whose job is removing noise?

It does work

The sampler, run on the little digit model we trained in L28:



On the right, one run followed down the chain. For most of the trajectory there is nothing to see - the structure appears late, in the last couple of hundred steps. That is characteristic, and it is a hint about where the model spends its capacity.

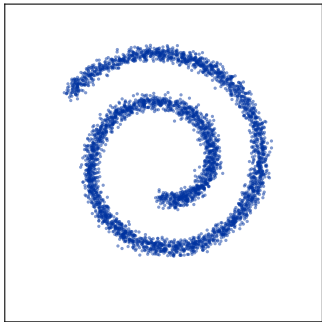
What the model actually learned

Two pixels instead of a million, so we can draw the whole thing.

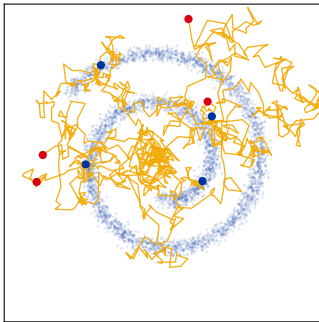
Shrink the image until you can plot it

An image is a point in pixel space. With two pixels, that space is a sheet of paper: the first pixel's brightness on one axis, the second on the other.

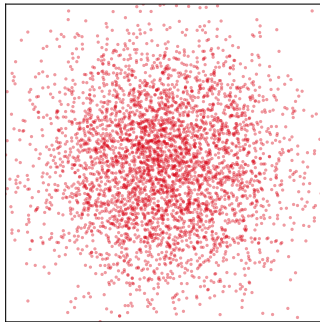
The data: two pixel values, one point



Noising one point = a random walk



Do it to every point ($t = 600$)



Real images occupy a thin, highly structured set inside a huge space. We fake that with a spiral: most of the plane is not data, and the data is curved and connected. **Every claim from here on is visible.**

Adding noise is a random walk with a leash

In the middle panel, one point takes 300 forward steps. Each step is

$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \varepsilon$$

which is a random jump *plus* a small pull back toward the origin.

A precise word on names. Pure Brownian motion has no pull - that is the variance-exploding process we rejected in L27. With the $\sqrt{1 - \beta}$ drift this is an **Ornstein-Uhlenbeck** process: it wanders, but it is on a leash, so it settles into $\mathcal{N}(\mathbf{0}, \mathbf{I})$ instead of drifting forever.

"Diffusion" is still the right name - the field takes it from **Sohl-Dickstein et al. (2015)**, who built the idea on non-equilibrium thermodynamics.

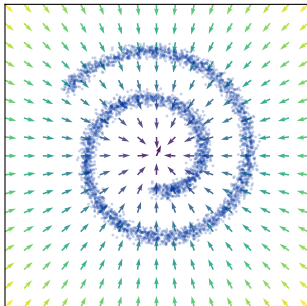
Generating an image is asking: given a point that has wandered off, **which way is back?**

The network is a direction field

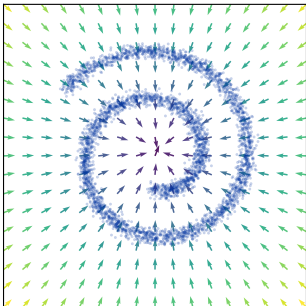
Ask ε_θ at every point on a grid, and plot which way it says to move:

What the network learned: a direction at every point, sharpening as $t \rightarrow 0$

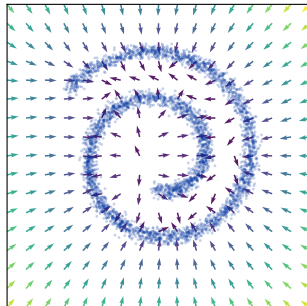
$t = 900$ (noise level high)



$t = 450$ (noise level medium)



$t = 50$ (noise level low)



High noise ($t = 900$): a broad field pointing at the data in general. It has no idea which arm you belong to, and it should not.

Low noise ($t = 50$): the field has resolved into the spiral's fine structure and points at the nearest arm.

That field has a name

Up to a constant, the direction the model gives is the **score** of the noisy data distribution:

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) = - \frac{\varepsilon_{\theta}(\mathbf{x}_t, t)}{\sqrt{1 - \bar{\alpha}_t}}$$

The gradient of the log-density: the direction in which data becomes *more likely*. Note the **minus sign** - moving away from the predicted noise is moving toward the data - and the $1/\sqrt{1 - \bar{\alpha}_t}$ scale.

Why this matters: predicting noise, predicting the score, and predicting the clean image are the same task in different coordinates. It also connects DDPM to a separate line of work (Song & Ermon, 2019) - and we use that connection at the end of this lecture.

This is also why conditioning on t was essential: one network has to represent all of these fields, and they are very different at $t = 900$ and $t = 50$.

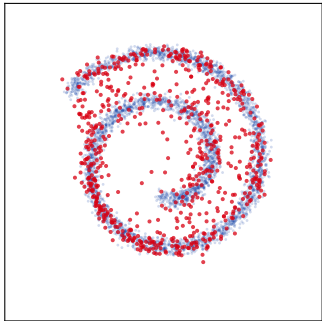
Why the noise comes back

The question from Section 1, answered.

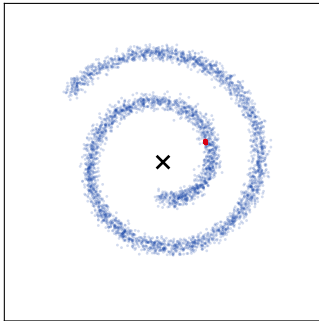
Run it three ways

Same trained model, same starting noise. Only the sampler changes.

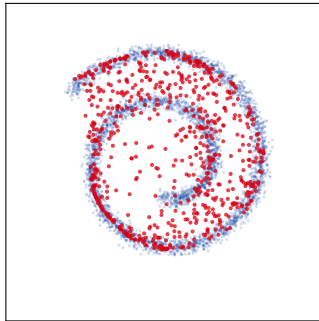
DDPM: keep the noise step
1000 steps, stochastic



Naive: just delete the noise
collapses to the mean



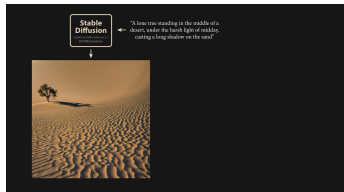
DDIM: rescale the step instead
25 steps, deterministic



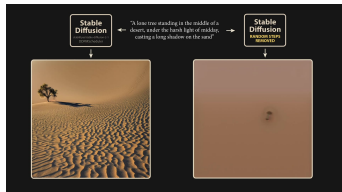
Left: the real sampler. Points land across the whole spiral. Measured spread: 0.68.

Middle: the noise term deleted. Every one of the 700 points lands in essentially the same place. Measured spread: **0.003**.

The same failure, on a real model



with the noise step



noise step deleted

Stable Diffusion, same prompt, same seed. Delete the noise term and you get a brown smear.

The spiral collapsed to a point. The image collapses to a **blur** - which is what the average of many images looks like.

Welch Labs / 3Blue1Brown (2025)

Why: the denoiser is being fed inputs it never saw

The network was trained at specific noise levels. At step t it expects its input to carry exactly the amount of noise the forward process puts there, $1 - \bar{\alpha}_t$.

Delete the $\sigma_t \mathbf{z}$ term and $\mathbf{x}_{t-1}$ comes out **cleaner** than σ_{t-1} demands. On the next call the network sees an input that is less noisy than anything in its training set.

Believing there is more noise than there is, it **removes too much** - and "remove as much as possible" points at the average of the training data. Repeat a thousand times and every trajectory ends up at that average.

The noise is not decoration - it is bookkeeping. It keeps each step's input at the noise level the next step was trained for. That, and not any deep need for randomness, is why it is there.

Keep that phrasing in mind: it tells us immediately how to build a *deterministic* sampler that does not blur.

One honest caveat about the toy

On the spiral, the collapsed points still land *on* the data - just all in one spot. So the toy shows lost **variety**, not lost **realism**.

In the space of real images, the average of many trees is not a tree at all. There the collapse leaves the set of realistic images entirely, which is why the output is a smear rather than one particular tree.

Where the analogy stops. A 2-D picture cannot show what "off the manifold" means in a million dimensions. Take the mechanism from the spiral, take the severity from the tree.

What this looks like when you get it wrong

You will implement this. Here is the same correctly-trained digit model, sampled three wrong ways - every fault below is at **sampling** time, not in the weights.

Same trained model, three sampling bugs. Two announce themselves. One does not.

Correct



Schedule mismatch: SILENT
different digits, still clean



Naive step skipping
every 40th step, no rescale



Wrong σ_t
3x too much noise



Skipping steps and **too much noise** are the fault we just diagnosed, in the other direction: the input reaches a noise level the network never trained on, and it breaks loudly.

The middle panel is the dangerous one. The wrong schedule gives clean, plausible digits - *different* ones, but nothing looks broken. A bug you can see costs an afternoon; a bug you cannot see ships.

Deterministic sampling

A thousand network calls per image is a problem worth solving.

The cost problem

The DDPM sampler runs the full chain: with $T = 1000$, that is **one thousand forward passes** of a large network per image.

For a while this was the reason diffusion stayed a research curiosity while GANs - one forward pass - shipped in products.

Do not attach "1000 steps" to Stable Diffusion. Shipping models use 20-50. The rest of this section is how they got there.

Predict first: we just spent a whole section proving that removing the noise ruins everything. So how can a *deterministic* sampler possibly work?

The resolution

The resolution

Do not *delete* the noise. **Rescale the step** so the noise level stays where the network expects it.

Our failed experiment kept the DDPM step size and dropped the noise, so the noise level fell out of sync. Instead, jump directly between consistent noise levels: predict the clean image, then re-noise it to exactly the level of the step you are moving to.

$$\hat{\mathbf{x}}_0 = \frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_\theta}{\sqrt{\bar{\alpha}_t}}, \quad \mathbf{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{\mathbf{x}}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \varepsilon_\theta$$

No random draw anywhere, yet every input is at a noise level the network trained on. This is **DDIM** - **Denoising Diffusion Implicit Models** (Song et al., 2020). **It needs no retraining**: same weights, different sampler.

It works, and it skips

Because consecutive $\bar{\alpha}$ values no longer have to be adjacent, you can take **25 steps instead of 1000** by striding through the schedule.

In the right-hand panel of the sampler figure: 25 steps, deterministic, measured spread 0.64 against DDPM's 0.68.

40x fewer network calls, same distribution, same weights. This is what made diffusion practical.

Two caveats worth stating.

1. The theory promises the same *distribution*, not the same image. A given starting noise gives different pictures under the two samplers.
2. The guarantee holds in the continuous limit. At 20 steps the discretization error is real and visible - very-low-step samples have a recognizable look.

STOCHASTIC DIFFERENTIAL EQUATION (SDE)

$$d\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x})] dt + g(t) d\bar{\mathbf{w}}$$

ORDINARY DIFFERENTIAL EQUATION (ODE)

$$d\mathbf{x} = \left[\mathbf{f}(\mathbf{x}, t) - \frac{1}{2} g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \right] dt$$

Resulting probability
distributions are the same!

$$x_{t-1} = x_t + a_t \epsilon_{\theta}(x_t, \sigma_t) + \eta w_t$$

DDPM UPDATE RULE (SIMPLIFIED)

$$x_{t-1} = x_t + b_t \epsilon_{\theta}(x_t, \sigma_t)$$

NEW UPDATE RULE (DDIM)

Where $a_t = (\sigma_{t'} - \sigma_t)$ $b_t = (\sigma_{t-1} - \sigma_t)$ $w_t \sim \mathcal{N}(0, I)$ $\eta = \sqrt{\sigma_{t-1}^2 - \sigma_t^2}$

See "Diffusion models from scratch, from a new theoretical perspective" by Chenyang Yuan

A note on where the theory came from

You will often see DDIM introduced through **stochastic differential equations**, as on the previous slide: the DDPM sampler is an SDE; a result from statistical mechanics (the Fokker-Planck equation) gives an **ordinary** differential equation - the *probability-flow ODE* - whose distribution at every time is the same.

Both update rules end up looking almost identical. The stochastic one has a random term; the deterministic one has a different step scaling instead.

That is not how DDIM was found. Song et al. (2020) derived it from a non-Markovian forward process, with no SDE anywhere.

The ODE connection came from Song et al. (2021), *afterwards*, and showed DDIM is roughly a first-order discretization of it.

Worth knowing both framings, and worth not confusing the order they happened in.

How would we even know which sampler is better?

We have been saying "better samples" for two lectures while judging by eye.

The standard answer is **FID (Fréchet Inception Distance)**: push real and generated images through a pretrained vision network, fit a Gaussian to each set of features, and measure the distance between them.

It rewards both **realism** and **variety**: a model producing one perfect image every time scores badly, which is exactly the failure we just watched.

What it misses: it says nothing about whether the image matches a *prompt*, it depends on the feature network, and it is unstable on small sample counts.

This closes the loop on L28: dropping the weights from the loss made the likelihood *worse* and FID *better*. Different objectives, and the field chose the one matching what people look at.

Recap

1. The DDPM sampler subtracts predicted noise, then adds fresh noise back.
2. Shrunk to two pixels, the trained network is a **direction field** - the score - that sharpens as $t \rightarrow 0$.
3. Delete the added noise and everything collapses to the data mean: blurry.
4. Because the denoiser is then fed inputs **cleaner than it was trained for**, and over-denoises.
5. DDIM fixes it by rescaling the step, not deleting the noise: deterministic, 25 steps, no retraining.
6. FID is how the field measures any of this.

Next: we can generate good images, quickly, from noise.

We still have no way to ask for anything. Every image so far has been whatever the model felt like producing.

L30 connects text to pixels.